

情感计算之人体行为识别

从检测到识别：HAR 全栈技术体系

2026 金砖赛 · 人工智能应用创新赛项

目录

#	主题	内容
1~3	理论基础	HAR 定义、双场景路线、YOLO 检测
4~6	跟踪与姿态	ByteTrack、COCO 骨架、姿态估计
7~9	行为识别	室内规则引擎、MMAction2 TSN、Kinetics-400
10~12	部署与服务	RESTful API、FastAPI、Web 前端
13~18	项目实操	环境搭建、室内管线、室外管线、API、前端

01 · 什么是 HAR

人体行为识别 (Human Action Recognition, HAR)

“ 通过分析图像或视频中的人体动作、姿态和运动模式，自动识别人类正在执行的行为类别 ”

技术交叉领域

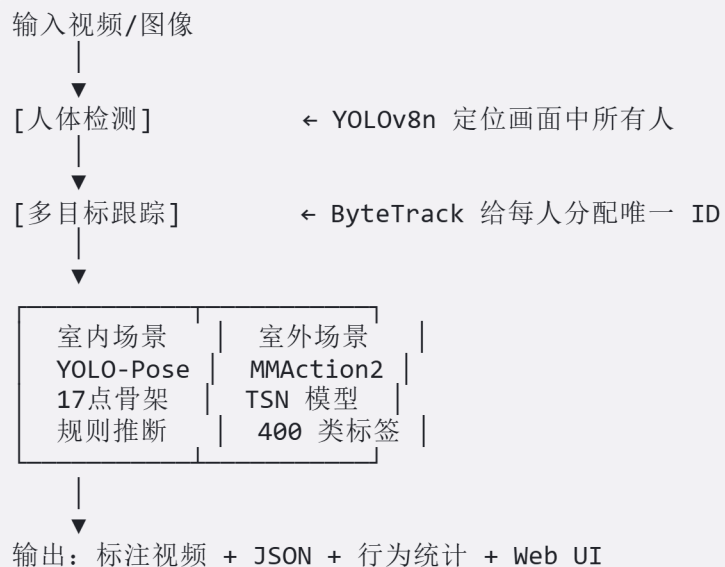
计算机视觉 + 深度学习 + 时序分析 + 情感计算 = HAR

核心应用：智慧教育 · 智慧安防 · 体育分析 · 人机交互 · 医疗康复 ·
自动驾驶

人体行为识别 (HAR) 基本流程



HAR 基本流程



HAR 应用场景

场景	应用示例
智慧教育	课堂行为分析：听讲、举手、写字、低头等状态监测
智慧安防	异常行为检测：跌倒、打架、奔跑、聚集等事件预警
体育分析	运动员动作识别与技术评估（Kinetics-400 覆盖大量体育动作）
人机交互	手势识别、体感游戏、AR/VR 中的自然交互
医疗康复	患者运动功能评估、康复训练进度监测
自动驾驶	行人意图预测（是否要过马路、奔跑等）

发展历程

1990s 萌芽期
手工特征+SVM

→

2014 双流期
Two-Stream CNN

→

2017 3D卷积期
C3D/I3D/SlowFast

→

2020 Transformer期
VideoMAE/TimeSformer

02 · 双场景技术路线

本课程的核心设计

场景	技术栈	输入	行为分类
室内	YOLOv8n-Pose + ByteTrack + 姿态规则	图片 / 视频	6 类：听讲、举手、写字、低头、站立、未知
室外	YOLOv8n + ByteTrack + MMAction2 TSN	仅视频	400 类 Kinetics 行为标签

设计理念

```
# 室内：行为-姿态映射明确 → 规则引擎（零训练数据）
infer_behavior_from_pose(kpts_17x3) → "raise_hand", 0.75

# 室外：行为复杂多变 → 深度学习（MMAction2 TSN）
recognizer.predict_frames(frames) → "dancing ballet", 0.85
```

“

室内靠**几何规则**，室外靠**深度学习**，各取所长

”

03 · YOLO 目标检测

什么是 YOLO

You Only Look Once — 单阶段目标检测算法 (2016)

“ 将目标检测视为回归问题，只需“看一次”图像就能同时预测位置和类别 ”

YOLO 系列演进

版本	年代	核心创新	参数量
YOLOv1~v3	2016-18	端到端检测 / FPN 多尺度	~62M
YOLOv5	2020	PyTorch 生态 + 工程优化	~7M (nano)
YOLOv8	2023	Anchor-Free + 解耦头 + C2f	~3.2M (nano) 
YOLOv11	2024	C3k2 模块 + 进一步轻量化	~2.6M (nano)

“ 本课程使用 **YOLOv8n**，CPU 即可实时运行 ”

YOLOv8 核心设计

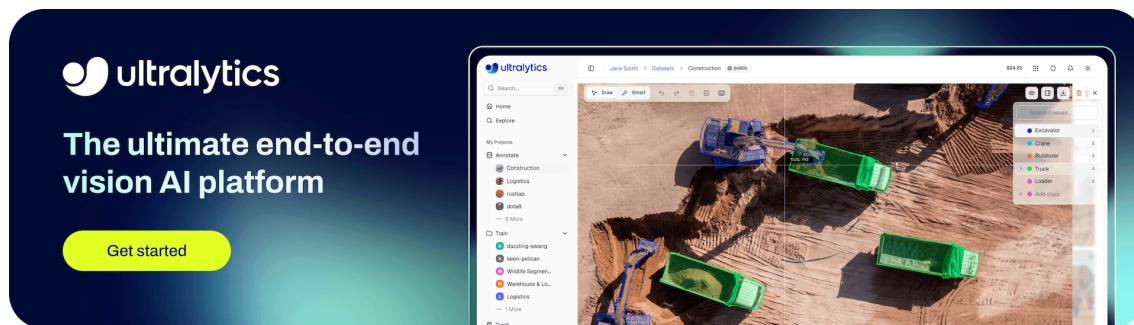
Anchor-Free 检测

传统 Anchor-Based:
预定义 9 种先验框
手动设计 anchor 尺寸
NMS 后处理复杂

YOLOv8 Anchor-Free:
直接从特征图预测 (x, y, w, h)
解耦分类头 + 回归头
简化设计, 泛化能力更强

解耦头 (Decoupled Head)

```
cls_out = cls_conv(features) # 分类分支: [B, num_classes, H, W]
reg_out = reg_conv(features) # 回归分支: [B, 4, H, W]
```



两个变体

- `yolov8n.pt` — 仅检测 (室外用)
- `yolov8n-pose.pt` — 检测 + 17 个 COCO 关键点 (室内用)

04 · ByteTrack 多目标跟踪

什么是 MOT

在视频连续帧中，**检测并持续跟踪**多个目标，为每个目标分配**唯一且稳定的 ID**

帧 1: [检测到 a, b 两人] → a→ID1, b→ID2
帧 2: [检测到 a, c 两人] → 哪个是 ID1? 哪个是新目标?
帧 3: [检测到 a, b, c 三人] → 保持 ID1/ID2 并分配 ID3

ByteTrack 核心创新: BYTE 策略

传统 MOT: 仅用高分框 ($\text{conf} > 0.5$) 匹配 → 遮挡时丢失目标

ByteTrack BYTE 策略:

高分框 ($\text{conf} > 0.5$) → 优先匹配已有 track

低分框 ($0.1 < \text{conf} \leq 0.5$) → 与未匹配 track 二次匹配
→ 被遮挡的人虽分数低，仍被关联到正确 track



Zhang et al., ECCV 2022 — 轻量级、高精度、可实时运行



ByteTrack 匹配流程

- ① 已有 track 用卡尔曼滤波预测下一帧位置
- ② 高分检测框 (D_high) 与所有 track 做 IoU 匹配
匹配成功 → 更新 track 位置
- ③ 低分检测框 (D_low) 与 ② 中未匹配的 track 做 IoU 匹配
匹配成功 → 更新 track 位置 (捞回被遮挡目标)
- ④ 仍未匹配的高分框 → 创建新 track (新人进入画面)
- ⑤ 连续 N 帧未匹配的 track → 删除 (已离开画面)

在行为识别中的关键作用

作用	说明
身份保持	同一人从进入画面到离开使用相同 ID
时序行为聚合	基于 track ID 收集连续行为标签 → 构建 segment
行为统计	统计每个 ID 各类行为的持续时间和占比

05 · 人体姿态估计

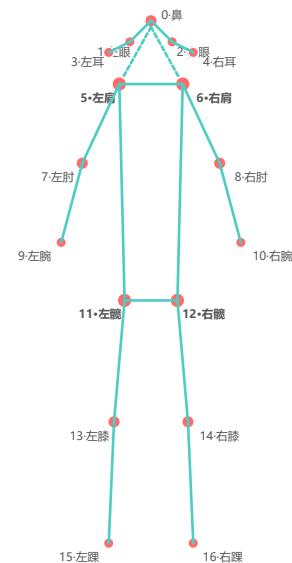
COCO 17 关键点体系

0: nose (鼻子)	
1: left_eye	2: right_eye
3: left_ear	4: right_ear
5: left_shoulder	6: right_shoulder
7: left_elbow	8: right_elbow
9: left_wrist	10: right_wrist
11: left_hip	12: right_hip
13: left_knee	14: right_knee
15: left_ankle	16: right_ankle

每个关键点格式: `[x, y, confidence]`

COCO 17 关键点人体骨架

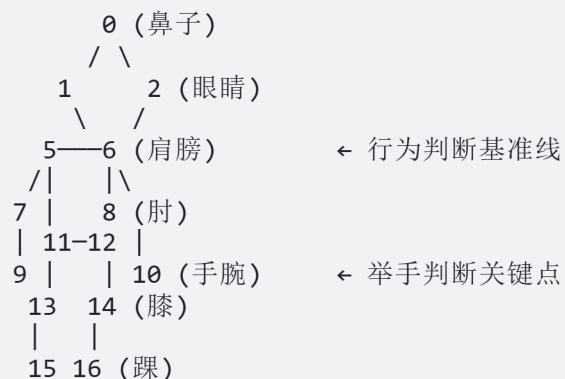
Common Objects in Context — 人体姿态估计标准关键点体系



每个关键点格式: `[x, y, confidence]`
红色圆点 = 关键点 | 青色连线 = 骨架连接 | 置信度 > 0.3 视为可见
共 17 个关键点 × 3 维 = 51 个值 (YOLOv8n-pose 输出)

骨架连接与行为判断

骨架连接定义



关键点在行为识别中的应用

关键点	行为判断用途
5, 6 (双肩)	基准线 : 判断其他部位是否高于/低于肩膀
9, 10 (手腕)	举手 : 手腕 $Y <$ 肩膀 $Y \rightarrow$ 举手
0 (鼻子)	低头 : 鼻子 $Y >$ 肩膀 $Y \rightarrow$ 低头
11, 12 (髌部)	站立 : 髌-肩距离估算躯干长度, 判定坐/站

06 · 室内行为：姿态规则引擎

规则引擎 vs 深度学习

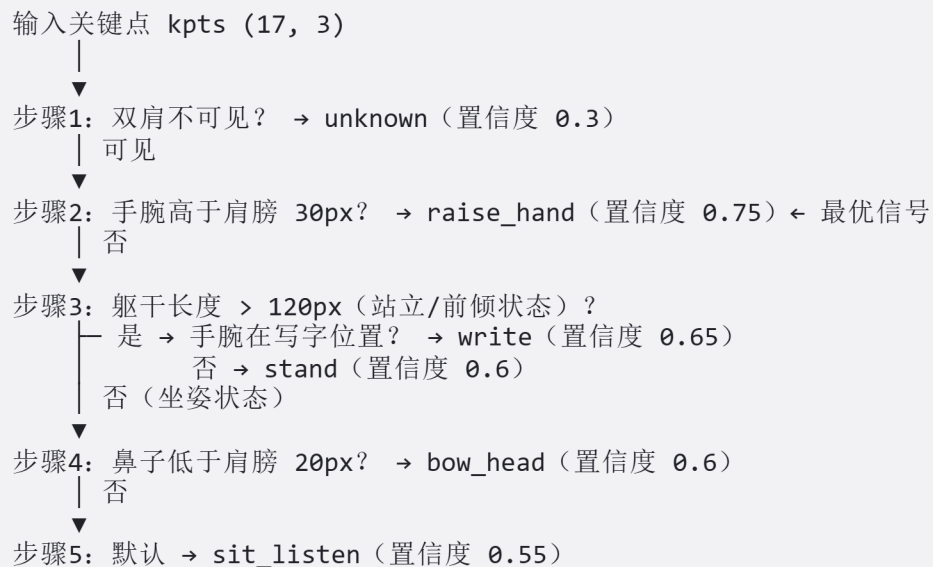
规则引擎	深度学习
零训练数据	需要大量标注数据
即时可用	需要训练时间
推理极快（纯 NumPy）	推理需要 GPU
结果可解释	黑盒决策
适合 明确的姿势-行为映射	适合模糊、多样的行为

六类室内行为

行为	英文	关键特征
sit_listen	坐着听课	躯干短（坐姿），无特殊姿势，默认状态
raise_hand	举手	手腕 Y 明显高于同侧肩膀 Y
write	写字	躯干较长（前倾），手腕低于肩膀
bow_head	低头	鼻子 Y 明显低于肩膀 Y
stand	站立	躯干较长（站立），无明显写字/举手特征
unknown	未知	双肩不可见或姿态无法判断

优先级判断链

行为判断遵循从特殊到一般的优先级链：



行为片段聚合

为何需要片段聚合

逐帧推断产生大量离散标签，需要组织成有时序意义的连续片段。

逐帧结果（平滑前）：

sit → sit → raise → sit → sit → sit → write → write

↑ 单帧跳变（噪声）

滑动窗口平滑（k=3）→ 多数字投票修正：

sit → sit → raise → raise → sit → sit → write → write

连续同行为合并 → Segments:

Segment 1: sit_listen (帧 0→1, 0.0~0.07s)

Segment 2: raise_hand (帧 2→3, 0.07~0.13s)

Segment 3: sit_listen (帧 4→5, 0.13~0.2s)

Segment 4: write (帧 6→7, 0.2~0.27s)

每个 Segment 包含：行为标签、置信度、起止帧号、时间戳、持续秒数

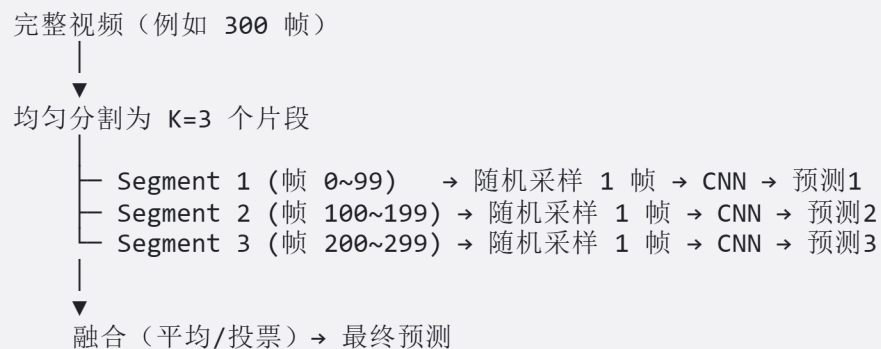
07 · MMAction2 与 TSN

什么是 MMAction2

OpenMMLab 开源视频理解工具箱 — 覆盖动作识别、时序动作定位、时空动作检测

TSN (Temporal Segment Network)

Wang et al., ECCV 2016 — 稀疏采样 + CNN 聚合的经典架构



TSN vs 3D CNN

特性	TSN	3D CNN (如 C3D、I3D)
建模方式	稀疏采样 + 2D CNN 聚合	密集 3D 卷积
计算量	低 ($K \times$ 单帧推理)	高 (连续帧立方体卷积)
长视频处理	天然支持	受限于显存
GPU 需求	低 (CPU 可推理)	高
适用场景	实时推理、CPU 部署	离线高精度分析

MMAction2 在本课程中的角色

特性	说明
模型	TSN + ResNet-50 主干
预训练	ImageNet + Kinetics-400 微调
推理设备	CPU (快速管线: 3 clips $\times$ 1 frame)
推理间隔	每 48 帧推理一次 (~ 1.6 秒), 间插复用
预热机制	启动时对整段视频预推理, 避免前 N 帧 unknown

MMAction2 推理管线

CPU 快速管线

```
_FAST_TEST_PIPELINE = [  
    dict(type='DecordInit'), # 视频解码器  
    dict(type='SampleFrames', clip_len=1, num_clips=3), # 采样 3 帧  
    dict(type='DecordDecode'), # 解码  
    dict(type='Resize', scale=(-1, 256)), # 短边缩放  
    dict(type='CenterCrop', crop_size=224), # 中心裁剪  
    dict(type='FormatShape', input_format='NCHW'), # 格式转换  
    dict(type='PackActionInputs'), # 打包输入  
]
```

默认管线采样 250 帧，快速管线仅 3 帧，CPU 可推理

两种推理模式

模式	scene (默认)	track
做法	对整个画面推理	裁剪每个人体区域单独推理
优点	全局上下文丰富	多人各自独立识别
缺点	多人同画面只能输出一个标签	速度更慢

08 · Kinetics-400 数据集

数据集概述

DeepMind, 2017 — 视频理解领域的"ImageNet"

属性	值
行为类别	400 类
视频总数	~306,000 段
每段时长	~10 秒
来源	YouTube 公开视频

行为类别示例

类别	示例行为
体育运动	playing basketball, swimming, surfing, skiing, archery
乐器演奏	playing guitar, playing piano, playing violin, playing drums
日常活动	cooking, eating, brushing teeth, reading
舞蹈	dancing ballet, belly dancing, breakdancing
社交互动	shaking hands, hugging, kissing
竞技格斗	boxing, wrestling, judo, fencing

Kinetics-400 在本课程中的使用

本课程的数据获取方式：

```
不下载 Kinetics-400 原始数据集
↓
直接使用预训练好的 TSN 推理权重
(tsn_imagenet-pretrained-r50_...pth)
↓
+ label_map_k400.txt 标签映射文件
↓
即可对任意视频进行 400 类行为推断
```

本课程聚焦**推理部署**，推理权重已提供，开箱即用

数据特点

-  类别极其丰富（400 类），覆盖广泛
-  视频来自真实场景（in-the-wild）
-  视频质量和拍摄角度参差不齐
-  每段仅约 10 秒，是“短视频片段”而非完整视频

09 · RESTful API 规范

REST 核心原则

原则	描述
资源 (Resource)	一切皆资源, URI 标识
无状态 (Stateless)	每个请求自包含
统一接口	标准 HTTP 方法操作资源
分层系统	支持负载均衡、缓存

HTTP 方法与 CRUD

方法	CRUD	HAR 项目示例
GET	Read	/api/v1/health — 健康检查
POST	Create	/api/v1/behavior/analyze — 上传分析
DELETE	Delete	/api/v1/jobs/{id} — 删除任务

HAR API 设计

HAR 行为识别 API 服务
http://localhost:8082

GET	/api/v1/health	→ 健康检查
GET	/api/v1/behaviors	→ 行为标签列表
GET	/api/v1/demo-samples	→ 内置 demo 列表
POST	/api/v1/behavior/analyze-demo	→ 一键分析 demo
POST	/api/v1/behavior/analyze	→ 上传文件分析（核心）
GET	/api/v1/jobs/{id}	→ 任务详情 + 结果
GET	/api/v1/jobs/{id}/result	→ 完整 JSON 结果
GET	/api/v1/media/{id}/original	→ 原始媒体文件
GET	/api/v1/media/{id}/annotated	→ 标注图片/视频
DELETE	/api/v1/jobs/{id}	→ 删除任务

API 异步任务与状态码

异步任务管理模式

行为分析是耗时操作，使用**异步任务管理**模式：

```
POST /api/v1/behavior/analyze
├─ 创建 job_id (UUID)
├─ 保存文件 → data/api/uploads/{job_id}/
├─ [立即返回] → { "job_id": "...", "status": "pending" }
└─ 后台线程池执行推理
    ├─ status → "running" (更新进度)
    └─ run_analysis(...)
        └─ status → "completed" (存入结果)
```

前端轮询: GET /api/v1/jobs/{id} → 查询状态 + 进度

状态码规范

状态码	含义	使用场景
200	成功	获取结果、健康检查
400	客户端错误	缺少参数、格式不支持
404	不存在	任务/文件不存在
422	参数校验失败	室外场景上传了图片

10 · FastAPI Web 框架

为什么选 FastAPI

特性	FastAPI	Flask
异步支持	✓ 原生 <code>async/await</code>	✗ 需额外插件
自动文档	✓ Swagger + ReDoc	✗ 需手动
数据验证	✓ Pydantic 类型验证	✗ 手动校验
性能	高 (异步 + Starlette)	中等 (同步 WSGI)
学习曲线	中等	低

最小应用

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello, HAR!"}

# uvicorn main:app --host 0.0.0.0 --port 8082
# 浏览器 → http://localhost:8082/docs → Swagger UI
```

“ HAR 分析是计算密集型异步任务，FastAPI 的原生异步支持更合适 ”

FastAPI 静态文件 + 前后端一体

```
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

if WEB_DIR.exists():
    app.mount("/assets", StaticFiles(directory=WEB_DIR), name="assets")

@app.get("/")
def index():
    return FileResponse(WEB_DIR / "index.html")
```

带请求体的分析接口

```
@app.post("/api/v1/behavior/analyze")
async def analyze(
    file: UploadFile = File(...),          # 上传的图片/视频
    scene: str = Form("classroom"),        # classroom / extracurricular
    max_frames: int = Form(None),          # 最大处理帧数
):
    # 校验格式 + 室外禁止图片 + 保存文件 + 后台推理
    return {"job_id": "...", "status": "running"}
```

11 · Web 前端基础

技术栈

纯 HTML + CSS + JavaScript (零框架依赖)

技术	作用
HTML5	页面结构：标题、卡片、上传区、结果展示
CSS3	样式：Flexbox 布局、响应式设计、动画过渡
JavaScript (Vanilla)	交互逻辑：上传、API 调用、轮询、结果渲染

页面模块

Header: 标题 + API 健康状态指示灯
快速试用: 3 个 Demo 卡片 (三等分布局)
室内视频 室内图片 室外 sports
上传区域: 拖拽或点击选择文件 场景选择: ☐ classroom ☐ extracurricular [开始分析]
结果展示: 摘要 + 行为统计 + 标注视频

前端关键交互

文件上传

```
const formData = new FormData();
formData.append("file", fileInput.files[0]);
formData.append("scene", selectedScene);

const resp = await fetch("/api/v1/behavior/analyze", {
  method: "POST", body: formData,
});
```

结果轮询

```
async function pollJob(jobId) {
  const poll = setInterval(async () => {
    const job = await fetch(`/api/v1/jobs/${jobId}`).then(r => r.json());
    updateProgress(job.progress_message);
    if (job.status === "completed") { clearInterval(poll); showResults(job); }
    if (job.status === "failed") { clearInterval(poll); showError(job.error); }
  }, 1000); // 每秒轮询
}
```

场景联动：室外仅支持视频 → 选图片时自动禁用提交

12 · 项目实操：环境搭建

环境准备

```
# 前提: Python 3.10 已安装
pip install -r requirements.txt

# torch 2.0.1 要求 numpy < 2
python -m pip install "numpy<2"

# MMDetection 环境验证
python scripts/setup_mmdetection.py --skip-pip
```

验证

```
python -c "import torch, ultralytics, cv2, yaml; print('OK')"
```

数据与模型说明

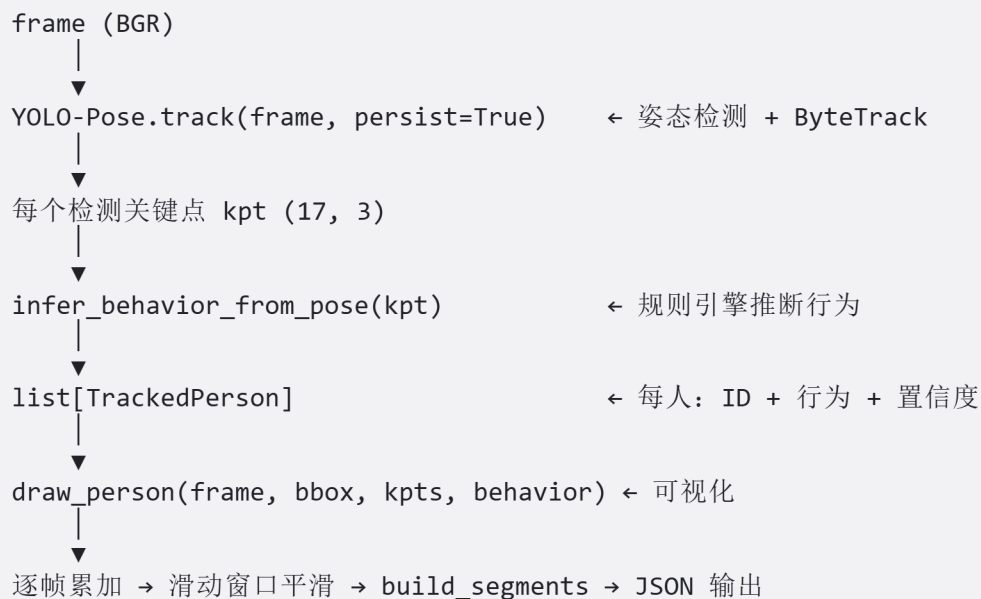
类别	路径	说明
室内 demo	data/demo/	推理演示视频与图片
室外 demo	data/demo/extracurricular/	含 sports.mp4
YOLO 权重	models/yolo/	yolov8n.pt 、 yolov8n-pose.pt
MMDetection 权重	models/mmdetection/	TSN Kinetics-400 预训练

项目结构一览

module_b_behavior/	
├── run_api.py	← Web + API 启动入口
├── run_infer.py	← CLI 推理入口
├── configs/	← classroom.yaml / extracurricular.yaml
├── src/	
│ ├── common/	← config、segments、video_io
│ ├── detect/	← person_detector (YOLO)
│ ├── track/	← PoseTracker / DetectTracker
│ ├── classroom/	← rules、pipeline (室内)
│ ├── extracurricular/	← mmaction、labels、pipeline (室外)
│ ├── pipeline/	← router (场景路由)
│ ├── visualize/	← draw (画框、骨架)
│ ├── analysis/	← keyframes (关键帧)
│ ├── api/	← app、schemas、jobs (FastAPI)
│ └── web/	← index.html + styles.css + app.js
├── data/demo/	← 演示数据 (已提供)
├── models/	← 预训练权重 (已提供)
├── scripts/	← 辅助脚本
└── output/	← 运行后生成

13 · 项目实操：室内管道

核心模块串联



输出产物

```
output_dir/
├── vis/{video_stem}_tracked.mp4    # 标注视频 (H.264)
├── results/{video_stem}_tracked.json # 分析结果 JSON
└── keyframes/{video_stem}/        # 关键帧
```

室内规则引擎核心代码

```
import numpy as np

def infer_behavior_from_pose(kpts: np.ndarray) -> tuple[str, float]:
    """输入 (17,3) COCO 关键点 → 输出 (行为标签, 置信度)"""
    ls, rs = 5, 6 # 双肩
    if not (kpts[ls,2] > 0.3 and kpts[rs,2] > 0.3):
        return "unknown", 0.3

    shoulder_y = (kpts[ls,1] + kpts[rs,1]) / 2

    # 手腕高于肩膀 30px → 举手
    for wrist, sh in [(9,5), (10,6)]:
        if kpts[wrist,2] > 0.3 and kpts[wrist,1] < kpts[sh,1] - 30:
            return "raise_hand", 0.75

    torso_len = abs(kpts[11:13, 1].mean() - shoulder_y)
    if torso_len > 120:
        wrist_below = (kpts[9,1] > shoulder_y + 40 or kpts[10,1] > shoulder_y + 40)
        return ("write", 0.65) if wrist_below else ("stand", 0.6)

    if kpts[0,2] > 0.3 and kpts[0,1] > shoulder_y + 20:
        return "bow_head", 0.6

    return "sit_listen", 0.55
```


14 · 项目实操：室外管道

MMAction2 集成流程



室外关键设计点

预热机制 `_bootstrap_scene()`

问题: `infer_interval=48`, 视频前 47 帧 `behavior = unknown`

解决: 启动时对整段视频先做一次 `MMAction2` 推理

- 获得初始 `scene_behavior`
- 第 0 帧开始就有正确的行为标签

推理间隔 `infer_interval=48`

每 48 帧调用一次 `MMAction2` 推理 (~1.6 秒间隔)

相邻推理之间的帧复用上一次结果

原因: `MMAction2` 单次推理约 200~500ms, 不能逐帧调用

室外 CLI 示例

```
python run_infer.py `
--media data/demo/extracurricular/sports.mp4 `
--output output/sports `
--scene extracurricular `
--max-frames 60
```

场景路由设计

```
def get_pipeline(scene="classroom", label_mode=None):
    """工厂函数：按场景返回对应 Pipeline"""
    if scene == "extracurricular":
        return ExtracurricularPipeline(label_mode=label_mode)
    return ClassroomPipeline()

def run_analysis(scene, media_type, input_path, output_dir, **kwargs):
    """统一分析入口"""
    pipeline = get_pipeline(scene, ...)

    if scene == "extracurricular" and media_type == "image":
        raise ValueError("室外场景仅支持视频")

    if media_type == "image":
        return pipeline.run_image(...)
    return pipeline.run_video(...)
```

场景	模型	行为来源	输入
classroom	YOLOv8n-Pose	规则引擎	图片 / 视频
extracurricular	YOLOv8n + TSN	MMAction2	仅视频

15 · 项目实操：API 服务化

启动服务

```
# Web 演示（推荐）  
python run_api.py --port 8082
```

```
=====
HAR 行为识别服务
Web UI:   http://0.0.0.0:8082
Swagger:  http://0.0.0.0:8082/docs
健康检查: http://0.0.0.0:8082/api/v1/health
=====
```

任务管理核心设计

```
class JobManager:
    def __init__(self, root, max_workers=1): # 单线程推理
        self._jobs: dict[str, Job] = {}
        self._lock = threading.Lock()
        self._executor = ThreadPoolExecutor(max_workers=1)
```

“ `max_workers=1` : 避免两个模型同时推理导致 CPU 过载 ”

API 端点全景

方法	端点	说明
GET	/api/v1/health	健康检查
GET	/api/v1/behaviors	行为标签列表
GET	/api/v1/demo-samples	内置 demo 列表
POST	/api/v1/behavior/analyze-demo	一键分析 demo
POST	/api/v1/behavior/analyze	上传文件分析
GET	/api/v1/jobs/{id}	任务详情 + 结果摘要
GET	/api/v1/jobs/{id}/result	完整 JSON 结果
GET	/api/v1/media/{id}/original	原始媒体文件
GET	/api/v1/media/{id}/annotated	标注图片/视频
DELETE	/api/v1/jobs/{id}	删除任务

测试 API

```
curl http://localhost:8082/api/v1/health
curl -X POST http://localhost:8082/api/v1/behavior/analyze `
-F "file=@test.jpg" -F "scene=classroom"
```

16 · 项目实操：前端平台

访问地址

启动 API 后访问 <http://localhost:8082>

前端功能

区域	功能
Header	标题 + API 健康状态指示灯
快速试用	3 个 Demo 卡片（室内视频 / 室内图片 / 室外 sports）
上传区	拖拽上传 / 点击上传文件
场景选择	classroom / extracurricular 单选
参数	最大帧数、可视化、关键帧
结果区	行为摘要 + 行为统计 + 标注视频播放

Demo 快速试用

ID	名称	场景	说明
classroom_video	室内视频	classroom	课堂视频分析
classroom_image	室内图片	classroom	单帧图片分析
extracurricular_video	室外视频	extracurricular	sports.mp4 一键分析

两套系统对比总结

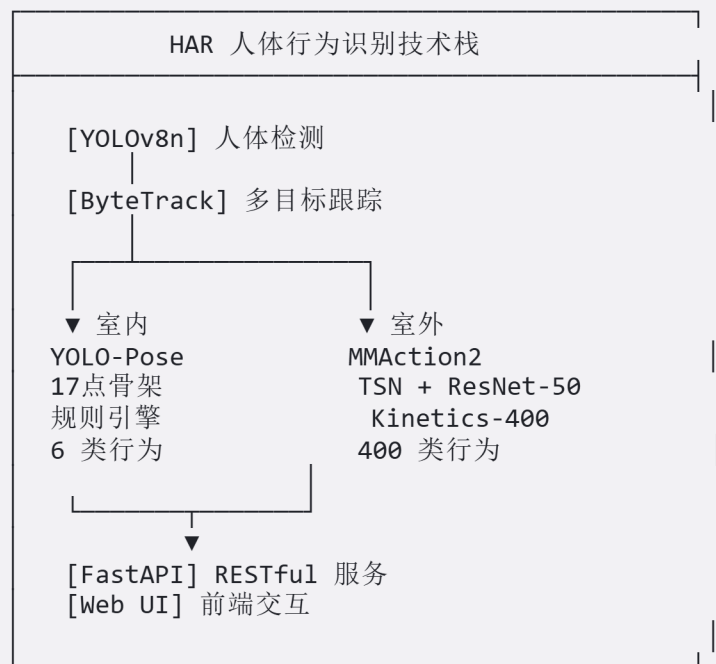
维度	室内（规则引擎）	室外（MMAction2）
核心方法	关键点几何	深度学习时序模型
模型	YOLOv8n-Pose	YOLOv8n + TSN ResNet-50
行为数	6 类	400 类
训练需求	零训练	预训练权重
推理耗时	~5ms/帧	~200-500ms/次（每 48 帧）
可解释性	高（明确知道为何判断）	低（黑盒决策）
适用输入	图片 + 视频	仅视频
适用场景	结构化室内	开放户外

设计理念

室内：行为-姿态映射明确 → 规则引擎（简单、快速、可解释）
室外：行为复杂多变 → 深度学习（泛化、覆盖广）

两种方法**互补而非对立**，根据场景特征选择最优方案

核心技术路线



课程总结

你已掌握的技能

环节	技能点
 理论	HAR 定义、双场景路线、Kinetics-400 数据集
 检测	YOLOv8 Anchor-Free 检测、YOLO-Pose 17 点姿态
 跟踪	ByteTrack BYTE 策略、多目标身份保持
 姿态	COCO 关键点体系、骨架连接、几何规则推断
 识别	室内规则引擎、MMAction2 TSN 时序分类
 部署	FastAPI RESTful API、异步任务管理、Web 前端

关键技术要点回顾

三个核心设计

1. **双场景分离** — 室内用规则（快速可解释），室外用深度学习（泛化覆盖广）
2. **CPU 可推理** — YOLOv8n (3.2M) + MMDetection 快速管线，无需 GPU
3. **端到端闭环** — 检测 → 跟踪 → 识别 → API → Web UI，一套命令启动

核心技术路线

YOLO 检测 → ByteTrack 跟踪 → 场景路由
├ 室内: YOLO-Pose → 规则引擎 → 6 类课堂行为
└ 室外: MMDetection TSN → Kinetics-400 → 400 类行为标签
↓
FastAPI 服务 → Web 前端 → 标注视频 + JSON 结果

参考资料

- Redmon, J. et al. (2016). *YOLO*. CVPR 2016
- Ultralytics (2023). *YOLOv8*. <https://docs.ultralytics.com/>
- Zhang, Y. et al. (2022). *ByteTrack*. ECCV 2022
- Wang, L. et al. (2016). *TSN*. ECCV 2016
- MMAction2 Contributors (2020). *OpenMMLab MMAction2*
- Kay, W. et al. (2017). *Kinetics-400*. DeepMind
- **FastAPI**: <https://fastapi.tiangolo.com/>
- **COCO 关键点**: <https://cocodataset.org/>

Thank You

准备好了吗？开始动手实践！

```
# Web 演示
python run_api.py --port 8082

# 室内 CLI
python run_infer.py --media data/demo/classroom01_clip_8min_1min.mp4 --scene classroom

# 室外 CLI
python run_infer.py --media data/demo/extracurricular/sports.mp4 --scene extracurricular --max-frames 60
```

“

📖 配套文档：

- 基础知识手册
- 项目实操手册（推理专用版）
- 技术架构说明

”